



DIVISION OF PHYSICAL SCIENCES AND MATHEMATICS

College of Arts and Sciences | University of the Philippines Visayas

Miagao, Iloilo, 5023, Philippines | Tel. No. (033) 513-7020

Email Address: psm.upvisayas@up.edu.ph

COURSE MATERIAL

Scanner

CMSC 124 Lab 1

Prepared by: Rene Andre Bedonia Jocsing

Published: September 04, 2026

Deadline: December 09, 2026

Lab 0 left you with a repository that builds on any machine, a `./run` script with one predictable interface, and a green check in the Actions tab. None of it does anything interesting yet. This activity is where your language starts to exist. Your first task is a **lexical scanner**, the component that reads raw source text and hands back the pieces a parser can work with.

A programming language needs at least three components before a machine can run it: a scanner, a parser, and an interpreter. We begin with the first one because everything downstream depends on it, and because it's the one component you can finish, test, and be certain about in four weeks. This activity follows Chapter 4, "Scanning," of Nystrom's *Crafting Interpreters*, which is the course's primary reference for the laboratory.

Background

Source code is text. A machine needs some way to turn that text into something it can act on. The scanner is the first stop in that pipeline. It takes characters and groups them into **tokens** that mean something in your language. You can think of the scanner as the front end of your source-to-execution pipeline.

Before you can scan anything, you have to decide what's worth scanning. Consider one line of Kotlin:

```
1 var myVariable = 4
```

You can chop that string into a huge number of substrings. You could form "my", "Vari", or "able = 4", yet none of them mean anything to Kotlin. Break it at the right places, though, and you get something useful:

```
1 var | myVariable | = | 4
```

Each chunk is a **lexeme**, the shortest run of characters that still means something in the language's grammar. Here, `var` declares a variable, `myVariable` identifies one, `=` assigns, and `4` is a number literal. Grouping characters into lexemes is called **lexical analysis**, or tokenization. Your scanner does it by walking the source one character at a time and deciding where each lexeme ends.

A lexeme on its own isn't enough for the phases that come later, so a scanner bundles each one with extra information and calls the result a **token**, composed of the following pieces:

The **token type** says what category the lexeme belongs to. The parser needs to know that `var` is a declaration keyword rather than an identifier that happens to spell “var”. Comparing raw strings everywhere would be slow and easy to get wrong. Classify once, in the scanner. Every phase downstream then gets to switch on a category instead.

The **literal value** is the one students most often misread. A token carries both the source text and the value that text stands for. Those are two different things. Scan `4` and the lexeme is the one-character string `4`, while the literal is the number `4`. Scan `"hi"` and the lexeme is four characters long, quotes included, while the literal is the two-character string `hi`. The lexeme is what the programmer typed. The literal is what it means. Keeping them apart is what lets your scanner translate an escape sequence like `\n` into an actual newline, since the lexeme still shows the backslash `\` and the `n` while the literal holds the character they denote. Tokens that aren’t literals, such as `var` or `=`, carry nothing in this field.

The **location**, at minimum a line number, is what lets you print an error a human can act on. It costs you one integer per token, but it’s the difference between “unexpected character” and “unexpected character on line 47.”

For the Kotlin line above, the tokens might come out as:

- Token: Type=VAR, Lexeme=“var”, Literal=null, Line=1
- Token: Type=IDENTIFIER, Lexeme=“myVariable”, Literal=null, Line=1
- Token: Type=EQUAL, Lexeme=“=”, Literal=null, Line=1
- Token: Type=NUMBER, Lexeme=“4”, Literal=4, Line=1

The rules that decide how characters group into lexemes are the **lexical grammar** of your language. For most languages, including whatever you’re about to invent, that grammar is simple enough to be a *regular language*, one you can recognize by reading the input straight through, left to right, remembering only which state you’re in and never how you got there: no counting, no backtracking, no memory of what came fifty characters ago. That restriction is what regular expressions are built to describe. A regex engine could do this whole activity for you in about four lines.

Which is why you can’t use one. We’re doing this from scratch, the way the cavemen of computer science once did, so **regular expressions are not allowed anywhere in your scanner**. Write the character loop. The point of the prohibition is that you’ll have built the machine underneath the abstraction. It’s a small enough machine that you can hold all of it in your head at once. That stops being true after this month, once you have a parser and an interpreter.

Learning Objectives

By the end of this activity, you should be able to:

- Explain the difference between a lexeme and a token, and say what a token carries besides its text
 - Implement a character-by-character scanning loop that can look ahead at characters it hasn’t consumed yet, without regular expressions
 - Handle multi-character operators, string literals, numeric literals, identifiers, keywords, and comments
 - Track source positions well enough to report a useful error
 - Separate program output on stdout from diagnostics on stderr, and return the exit code the run contract asks for
 - Design the first version of your own language’s lexical grammar and defend the choices behind it
 - Write regression tests for output that only your group can define, and keep them green in continuous integration
-

Task

Build a scanner for the language your group is designing, reachable two ways from the repository you set up in Lab 0.

The tested path is `./run --tokenize <path-to-source-file>`, which scans one file and prints its token stream to stdout. The demonstrated path is `./run` with no arguments, which starts a read-eval-print loop (REPL) that scans each line you type and prints its tokens. The REPL is what you'll drive during defense. The file path is what the test harness drives on every push.

Alongside the code, you'll write the first draft of your language specification in your repository's `README.md`, and commit a folder of scanner tests with the expected output you decided on.

Required Features

Token Coverage

Your scanner has to recognize, at minimum:

- Single-character tokens: grouping characters, arithmetic operators, and whatever punctuation your language uses
- Multi-character operators such as `==`, `!=`, `<=`, and `>=`, or your language's equivalents
- String literals, including your decision about escape sequences and multi-line strings
- Numeric literals, both integers and decimals
- Identifiers, and the reserved keywords that identifiers are not
- Comments, which are scanned and discarded rather than emitted as tokens
- Whitespace and newlines, discarded but counted, so line numbers stay honest
- An end-of-file token, so the parser in Lab 2 has something unambiguous to stop at

Those are categories rather than names. The names are yours. Writing them down is the first concrete thing your language design produces, so do it before you write a line of scanner code.

```

1 LEFT_PAREN RIGHT_PAREN LEFT_BRACE RIGHT_BRACE
2 PLUS MINUS STAR SLASH
3 EQUAL EQUAL_EQUAL BANG BANG_EQUAL LESS LESS_EQUAL
4 IDENTIFIER STRING NUMBER
5 VAR PRINT IF ELSE WHILE TRUE FALSE NIL
6 EOF

```

That's an example, not a requirement. Your language may have no braces, or five keywords where the example has eight, or words where those have symbols. What matters is that every token your scanner can emit appears in the list exactly once, that the list lives somewhere your parser can refer to next month, and that you can say why each entry earns its place.

The Token Stream Format

The format is yours to choose, but each printed token has to carry its type, its lexeme, its literal value where one exists, and its line number. One token per line keeps failures readable. The reference implementation from *Crafting Interpreters* prints something like `Token(type=NUMBER, lexeme=4, literal=4.0, line=1)`. That structure is a perfectly good choice.

Whatever you pick, freeze it before you write your first test file, and write it down in your `README.md`. Every expectation you commit is compared verbatim, so a cosmetic change to the format later means updating every `.expected` file in the folder. That's not a world-ending disaster, but it's an afternoon you didn't have to spend.

The token stream also has to be **deterministic**, which is easy to miss and painful to debug. Two runs on the same input must print the same bytes in the same order. If you build your keyword table with a hash map and then iterate over the map anywhere in your output path, some languages will hand you a different order each run and your tests will fail at random.

Error Reporting

A scanner meets malformed input constantly. Two cases you must handle are a string literal with no closing quote and a character that can't begin any lexeme in your language.

When either happens, report the problem on `stderr` with the line number, keep scanning so you can report more than one problem per file, and exit **65** once the file is done. Nothing about a rejected file belongs on `stdout`. Exit 0 is reserved for a clean scan. 65 comes from the old Unix `sysexits.h` convention that `jlox` and `clox` in *Crafting Interpreters* both follow, where it means the input was rejected before anything ran.

Language Specification

Start designing the language itself now. How is a variable declared? Which words are reserved? Does whitespace matter to your scanner, or is the language bracketed like the C family? How are comments written, and are nested or docstring-style comments allowed? How do loops look? Most importantly, why? A design you can't justify is a design you'll abandon under pressure in Lab 4.

Document all of this on the landing page of your repository by editing your `README.md` to follow the CMSC 124 language specification [template](#), published alongside this manual on the course site. The template lists every section your specification needs and says which activity each one becomes due in. Lab 1 owns the creators, overview, host language, running it, file extension, lexical structure, whitespace, token output format, errors, and design rationale sections. The rest wait for the activity that forces the decision. Two rules are not negotiable. The language must be **dynamically typed**. Its *prototype* must be **feasible in three months**.

Don't agonize over getting the design right on the first pass. Changing your scanner later is cheap as long as you understand the code you wrote. Shifting requirements are normal on long software projects. Reading, scaling, and maintaining code is the job. It's also why vibe coders won't replace software engineers.

The REPL

With no arguments, `./run` reads a line at a time and prints the tokens for it. Errors print and the prompt returns. A bad line must not kill the session. The harness never touches this path, so the REPL is graded by demonstration and by the questions that follow it.

The Run Contract

Lab 0 fixed the interface. It doesn't change:

- `./build.sh` builds your interpreter once from a clean checkout.
- `./run <path>` runs it on one file. Program output goes to `stdout`, diagnostics go to `stderr`, exit 0 means the file went through clean, 65 means your scanner rejected it before running any of it, and 70 means the scan started and then died partway. Exit 70 won't come up until Lab 3.

This activity adds one wrinkle. From Lab 4 onward, plain `./run <file>` means "execute this program," so the token dump needs a home that doesn't collide with it. That home is a flag:

```
1 ./run --tokenize tests/lab1/keywords.mylang
```

Pick the exact spelling yourself and record it in your `README.md`, but keep the flag alive for the rest of the semester. It's what lets your Lab 1 tests keep passing in November, when `./run` on its own has grown into a full interpreter. Every pipeline stage you build gets a flag like this. That's how every earlier stage stays inspectable and testable after the stage above it lands.

The harness passes exactly one flag, so `--tokenize` has to do the whole job on its own. `"--tokenize --verbose"` in a manifest is one argument with a space in it. Your program will reject it.

Writing Tests

Nobody grades a scanner by reading its source and imagining what it prints. Your repository has to hold the evidence, and continuous integration (CI) has to re-check that evidence on every push. This is what the shared test harness from Lab 0 is for: [cmisc-124-harness](#). It calls your `./run` on each test file you committed and compares what comes back against what you said should come back. It doesn't know your grammar, your token names, or your host language.

The Manifest

Create `tests/lab1/` and declare that folder's conventions in `tests/lab1/manifest.json`:

```
1 {  
2   "ext": ".mylang",  
3   "flag": "--tokenize",  
4   "mode": "sidecar"  
5 }
```

Change `ext` to the extension you gave your language's source files, and `flag` to whatever you called your tokenize flag. Get `ext` wrong and the harness reports that it found no test files at all, which is a red X that has nothing to do with your scanner. Fields you leave out fall back to defaults. That's why `run_entrypoint` isn't here. `./run` is already the default.

Why Sidecar Mode for This Activity

The harness supports two ways of recording what a test expects. In **sidecar** mode, expectations live in separate files next to the test. In **inline** mode, they live in comments inside the test file itself, the `// expect:` convention from *Crafting Interpreters*. Inline is nicer to read and it's what you'll use from Lab 3 onward, but it's the wrong tool here, because every token your scanner prints carries a line number. Your error messages and Lab 2's parser both need to know where things were. And an inline annotation is a comment written inside the very file your scanner reads, which makes the annotation itself a line of that file, counted like any other.

Put those two issues together and the test interferes with what it's testing. Each expectation you write above a piece of code pushes that code further down the file, so the line number you just recorded stops being true the moment you record it.

Here's what a person naturally writes, testing two declarations:

```
1 var x  
2 // expect: TOKEN(var, line=1)  
3 // expect: TOKEN(x, line=1)  
4 var y  
5 // expect: TOKEN(var, line=2)  
6 // expect: TOKEN(y, line=2)
```

The last two expectations are wrong. `var y` is the second line of code but the fourth line of the file. The two annotations describing `var x` sit between them. Your scanner reports 4, your test insists on 2, and you spend an afternoon debugging a scanner that was right the whole time.

Writing it correctly means putting `line=4` beside code that plainly reads as the second line, then renumbering every expectation below any case you later insert. Nobody does that correctly twice.

Trailing annotations don't rescue you either. A comment at the end of a line adds no lines of its own, the usual dodge, but a scanner emits one token per lexeme, so a single line of source produces several

lines of output and needs several expectations. Once you need more expectations than you have source lines, they stack up and the pushing starts again.

Sidecar mode keeps expectations out of the source entirely, so the line numbers you wrote stay the line numbers your scanner sees.

Writing One Test

A test is a source file plus the output you expect from it. Start with the source, `tests/lab1/keywords.mylang`:

```
1 var greeting = "hello"
2 print greeting
```

Run your scanner on it once by hand. Read the output, all of it, and decide whether it's what you meant. This is the only moment in the process where a human judges the answer, so don't skim it. Once you believe it, save it verbatim as `tests/lab1/keywords.expected`:

```
1 Token(type=VAR, lexeme=var, literal=null, line=1)
2 Token(type=IDENTIFIER, lexeme=greeting, literal=null, line=1)
3 Token(type=EQUAL, lexeme==, literal=null, line=1)
4 Token(type=STRING, lexeme="hello", literal=hello, line=1)
5 Token(type=PRINT, lexeme=print, literal=null, line=2)
6 Token(type=IDENTIFIER, lexeme=greeting, literal=null, line=2)
7 Token(type=EOF, lexeme=, literal=null, line=2)
```

That test expects a clean scan, so it needs nothing else. For a test that should be rejected, add an exit-code file. `tests/lab1/unterminated.mylang` holds a string with no closing quote:

```
1 var broken = "no closing quote
```

Your scanner prints a diagnostic on stderr, so stdout is empty and `tests/lab1/unterminated.expected` is an empty file. Then `tests/lab1/unterminated.exit` contains one line:

```
1 65
```

Be clear-eyed about what that test proves. In sidecar mode, the harness checks stdout and the exit code. It doesn't check stderr. It only echoes stderr back to you when a test fails for another reason. So this test proves your scanner rejected the file, not that it explained itself well. The quality of your error messages is graded at defense, by a human, because humans will be the ones deciphering your error messages.

Understand what these tests are and aren't. Nobody can tell you the right token names for a language you invented, so there's no external answer key to compare against. This is a **regression check**. Your scanner's behavior today goes against the behavior your own group signed off on earlier. It catches the refactor that renames `STRING` to `STR`, and it catches the fix that turns a rejection into a crash. A token vocabulary that was wrong from the start it can't catch. That's what defense is for.

Organizing the Folder

Discovery is recursive and pairing is by filename stem, so `foo.mylang` looks for `foo.expected` next to it. Nothing needs registering anywhere. Add your hundredth test by adding files:

```

1  tests/lab1/
2    manifest.json
3    keywords.mylang
4    keywords.expected
5    numbers.mylang
6    numbers.expected
7    operators.mylang
8    operators.expected
9    strings/
10   escapes.mylang
11   escapes.expected
12   unterminated.mylang
13   unterminated.expected
14   unterminated.exit      <- contains: 65

```

A test file with no matching `.expected` fails. You can't lose a case by forgetting its expectation. The reverse, an `.expected` left over after you renamed its test, is reported as a warning. It won't fail the run, but the harness flags it, because a stale expectation is otherwise invisible.

Two mechanical details here will save you a confusing hour. A missing or extra trailing newline at the end of the file is forgiven, but whitespace inside a line isn't, so don't reformat your token output casually. And each test file gets 15 seconds before the harness kills it and reports a timeout, generous for a scanner until your test inputs get large, so ensure that each test stays atomic and well-scoped.

Running the Harness Yourself

Push-and-pray is a slow way to work. Run the same check locally first:

```

1  curl -sSL https://raw.githubusercontent.com/WhiteLicorice/cmsc-124-harness/v1.1/run_tests.py
   ↪ -o run_tests.py
2  ./build.sh
3  python run_tests.py tests/lab1

```

On Linux and macOS, use `python3` instead of `python`. On Windows, run this from Git Bash. Output looks like this:

```

1  [PASS] tests/lab1/keywords.mylang
2  [PASS] tests/lab1/strings/unterminated.mylang
3
4  2/2 tests passed.

```

And when something breaks, you get the difference:

```

1  [FAIL] tests/lab1/keywords.mylang
2      stdout mismatch:
3      --- expected
4      +++ actual
5      -Token(type=STRING, lexeme="hello", literal=hello, line=1)
6      +Token(type=STR, lexeme="hello", literal=hello, line=1)
7
8  1/2 tests passed.

```

Don't commit `run_tests.py`. Fetch it when you need it, the way CI does, and add it to your `.gitignore` if you keep forgetting.

Continuous Integration

Your workflow from Lab 0 already builds the project and runs `tests/lab0`. Add this activity's folder to it, on its own line, in the same repository:

```
1 - run: python3 run_tests.py tests/lab0
2 - run: python3 run_tests.py tests/lab1
```

Keep the `lab0` line. Every folder you've ever committed stays in the workflow for the rest of the semester. All of them have to stay green. That's the point of the flag on `./run`. A change to the parser in Lab 2 that breaks the scanner announces itself in the Actions tab within a minute. Surfacing it during your Lab 5 defense is the alternative. Giving each folder its own step means the log shows which activity broke, which a single loop over `tests/*` would hide.

The rest of the workflow, including the toolchain setup for your host language and the `curl` that fetches the harness, is unchanged from your Lab 0 manual. The pin stays until I announce otherwise through proper channels.

Book your defense only after the Actions tab shows a green check on the commit you intend to defend, because I'll be checking your repository upon booking, and I'll be rejecting repositories that are unprepared for presentation.

Implementation Notes

The Scanning Loop

The core is smaller than it looks. Keep two indices into the source: `start`, marking where the lexeme you're currently building began, and `current`, marking how far you've read. Everything between them is the lexeme so far. Then repeat: reset `start` to `current`, read one character, work out which lexeme it might start, consume the rest of that lexeme, and emit a token. Stop at the end of the input, then emit your end-of-file token.

```
1 function scanTokens():
2     while not atEnd():
3         start = current          // this lexeme begins here
4         scanToken()
5     addToken(EOF)
6     return tokens
7
8 function scanToken():
9     c = advance()
10    if c is "(" : addToken(LEFT_PAREN)
11    if c is "+" : addToken(PLUS)
12    if c is "=" :
13        if match("=") : addToken(EQUAL_EQUAL)
14        else          : addToken(EQUAL)
15    if c is " " or "\t" : return          // discard, emit nothing
16    if c is "\n" : line = line + 1; return
17    if c is a digit : number()
18    if c starts an identifier : identifier()
19    otherwise : reportError(line, "Unexpected character.")
```

Three helpers carry most of the weight, used in the scanner at the right moment:


```

1  function advance():
2      c = source[current]
3      current = current + 1
4      return c                // consumes
5
6  function peek():
7      if atEnd() : return "\0"
8      return source[current]   // does NOT consume
9
10 function match(expected):
11     if atEnd() or source[current] != expected : return false
12     current = current + 1
13     return true              // consumes only on a match

```

What peek does, reading a character you haven't consumed yet so you can decide what the current lexeme is without committing to it, is called **lookahead**. It's the scanner's only way to answer a question like "is this = the whole token, or the front half of ==?", because the answer lives one character further on than the character you're holding. A scanner that needs to see one character ahead has one character of lookahead, all this activity requires, unless your language requires two characters or three characters and so on. Your parser next month will want the same trick at the token level.

`match` is just peek plus a conditional advance, bundled because that pair comes up constantly.

Operators That Share a Prefix

`=` and `==` start the same way, so you can't decide on the first character alone. Read `=`, then look ahead. If the next character is also `=`, consume it and emit the two-character token, otherwise emit the one-character token. That's the `match("=")` line in the sketch above.

This is the **maximal munch** principle. The scanner takes the longest lexeme that matches, so `<=` must be checked before `<`, and a language with `!`, `!=`, and `!==` has to check longest-first. Check `!` first and you'd emit `!` followed by `==` for input the programmer wrote as one operator. The parser would fail on a line that was perfectly legal.

Identifiers and Keywords

Don't try to recognize keywords character by character. Scan the whole run of identifier characters first, then look the finished text up in a table of reserved words. A match means keyword. No match means identifier:

```

1  function identifier():
2      while peek() is a letter, digit, or "_":
3          advance()
4      text = source[start .. current]    // the finished run
5      type = keywords[text]             // table lookup
6      if type is absent : type = IDENTIFIER
7      addToken(type)

```

That's one line of logic. It's also the reason `variable` doesn't scan as `var` followed by `iable`. Maximal munch again. Consume everything that could belong to the identifier, and only then ask what you've got. Do it the other way around, checking for keywords as you go, and every identifier beginning with a reserved word breaks. `iffy` becomes `if` plus `fy`. You'll spend an hour debugging it.

Numbers and Strings

For numbers, consume digits, then decide what a `.` means. If a digit follows the dot, it's part of the number. If not, the dot is its own token, so `3.toString` and `3.14` both scan sensibly. Decide whether a

leading or trailing dot is legal in your language and write the decision down.

For strings, consume until the closing quote. Two decisions are yours: whether a string may span lines, and whether escape sequences like `\n` and `\"` exist. If they do, this is where you translate them. The literal value your token carries should be the string value itself. The source text stays in the lexeme. If you skip escapes for now, say so in your `README.md`. Leaving it ambiguous invites questions.

Comments and Line Counting

Comments look like operators until they don't. In a C-style language, `/` is division unless the next character is another `/`, in which case you discard the rest of the line. Block comments and nested block comments are more work and entirely optional, but if your language claims to support them, your tests should prove it.

Increment your line counter in exactly one place if you can manage it, usually where you consume a newline. Scattered increments are how a scanner ends up reporting line 3 for a token on line 4, and multi-line strings and block comments are where that bug hides.

Common Pitfalls

- Reaching for a regular expression. It's prohibited here, and the point of the prohibition is that you'll understand the machine underneath afterward.
- Off-by-one line numbers. Newlines inside strings and block comments are the usual culprits.
- An unterminated string that runs to the end of the file. Your loop must notice the end of input, not read past it.
- Emitting `!` and `=` where the source said `!=`. Check the longest operator first.
- Looking up keywords before the identifier is fully scanned.
- Forgetting the end-of-file token. Lab 2's parser needs it to know when to stop. Adding it later means regenerating every `.expected` file.
- Nondeterministic output. Anything that iterates a hash map on the way to stdout can reorder itself between runs and turn your test suite into a coin flip.
- Diagnostics on stdout. They belong on stderr, and the harness compares stdout byte for byte.
- Windows line endings. If Git is converting line endings on checkout, a `.expected` file committed with CRLF can mismatch output produced with LF on the CI runner. A `.gitattributes` with `* text=auto eol=lf` settles it.
- Committing your test expectations without reading them. An expectation you didn't scrutinize is a bug you've promoted to a requirement.

Testing Strategy

Aim for coverage by category, and write the test at the same time as the feature it covers. Concretely, `tests/lab1/` needs at least one file for each of these:

- Keywords and identifiers, including an identifier that begins with a keyword, such as `variable` or `iffy`
- Single-character operators and punctuation
- Multi-character operators, with the single-character version of each nearby
- Integer and decimal numbers, including a number followed by something that isn't a digit
- Strings, including one that's empty, and escapes if your language has them
- Comments, including one at the end of a file with no trailing newline
- Multi-line input, to prove your line numbers survive
- An empty file, which should produce only the end-of-file token

Then the rejections, each with its own `.exit` file containing 65:

- An unterminated string

- A character that can't start any lexeme in your language
- Whichever third case your specific design makes possible

The last one is a question for you to answer. If your language has nested block comments, an unclosed one belongs here. If it has a character escape syntax, an invalid escape does.

Deliverables

Your group's GitHub repository, presented during appointments, with a clean incremental commit history showing who wrote what, must contain:

1. Your scanner source, documented where the code doesn't speak for itself
2. `build.sh` and `run` at the repository root, executable, with `run` supporting both the `tokenize` flag and the no-argument REPL
3. `tests/lab1/` with `manifest.json`, your test sources, and every `.expected` and `.exit` file they need
4. `.github/workflows/test.yml` running the harness against `tests/lab0` and `tests/lab1`, green on the commit you defend
5. A `README.md` holding your language specification, following the specification template's section order, and recording your token output format, your `tokenize` flag, and your decisions about strings, comments, and numbers

Then, each member submits, individually after the laboratory defense through email: a short `reflection.txt` covering what broke, what you fixed, and what you learned, plus a short `peer.txt` with your honest assessment of how your groupmates, including yourself, worked during the activity. Adhere to the following subject line: [CMSC 124 Lab] Lab 1: LastName, Initials, for example: [CMSC 124 Lab] Lab 1: Sanchez, SM. Include a link to your group's GitHub repository in the email. If even one member of a group fails to submit their individual deliverables, no final grade for the activity may be released for all members of the group.

Academic Honesty

Using large language models (ChatGPT, Claude, DeepSeek, and the rest) to generate wholesale vibecoded submissions is cheating. It's against the university's code of ethics and it's subject to failure in the course and harsh disciplinary action. The defense exists partly to make this visible. You'll be asked to explain the code you committed and to take responsibility for it. A scanner you didn't write is very hard to explain under questioning.

Important Dates

Progress reports and the laboratory defense may be booked only during the dates and hours defined in the syllabus. Book ahead of time through the booking page on the course site. Walk-ins during these periods aren't entertained.

Activity	Monday	Tuesday
Week 1 Progress Report	Sep 7	Sep 8
Week 2 Progress Report	Sep 14	Sep 15
Week 3 Progress Report	Sep 21	Sep 22
Week 4 Laboratory Defense	Sep 28	Sep 29

I have to verify appointments before they count. If one of these dates falls on a holiday or a suspension of classes, the syllabus lets you submit that week's progress report or defense as a recording instead. See the syllabus for the format and the subject line.

Progress reports build toward the defense. Week 1 wants design notes and a working prototype: your token list, your language's first design decisions, and a scanner that handles single-character tokens. Week 2 wants the scanner handling every category above, with tests committed as you go. Week 3 wants the complete implementation, green CI, and your documented quirks. The one-month period is buffer built for your protection. The groups who use it are the ones who don't spend the final week discovering that their line numbers were wrong all along.

Grading Rubric

Criteria	Excellent (90-100%)	Good (75-89%)	Fair (60-74%)	Poor (0-59%)
Implementation Correctness (25%)	Every required feature of the activity works, including edge cases; the run contract is honored exactly, with program output on stdout, diagnostics on stderr, and the right exit code for each class of failure.	Core features correct with minor gaps in fringe cases; streams and exit codes right.	Core features work, but several requirements are missing or wrong; stream separation or exit codes are inconsistent.	Fails the activity's core requirement, ignores the run contract, or crashes with host-language errors on malformed input.
Testing and CI (20%)	Committed tests cover every required feature and its failure cases; expectations were reasoned out rather than pasted from output; every lab folder is green in CI on the defended commit.	Good coverage of the main features with at least one failure case; CI green.	Only happy paths tested, or an earlier lab folder left failing.	Almost no tests, no manifest, or no working workflow.
Code Engineering (10%)	Clean separation between pipeline stages; shared logic written once; constructs idiomatic for the host language; comments explain why rather than what.	Mostly clean structure with minor duplication or naming problems.	Monolithic functions; repeated logic; one stage leaking into another.	Unreadable, or the code contradicts the documented language design.
Collaboration (20%)	Professional Git usage: atomic, semantic commits from every member spread across the accomplishment period; tests and features arrive together; evidence of review or pair programming.	Consistent use of version control; adequate commit messages; evidence of a structured team workflow.	Inconsistent Git usage; large code dumps instead of incremental progress; vague commit messages.	Minimal use of version control; repository lacks history, or history was rewritten or crammed.
Technical Defense (25%)	Every member traces their own code over fresh input, handles what-if questions, and defends the language design decisions behind the implementation.	Clear explanation of the implementation; both members participate meaningfully; logic delivery is sound.	Unclear explanations; uneven participation; struggles with code-tracing questions.	Unprepared; cannot explain the pipeline or defend design decisions.